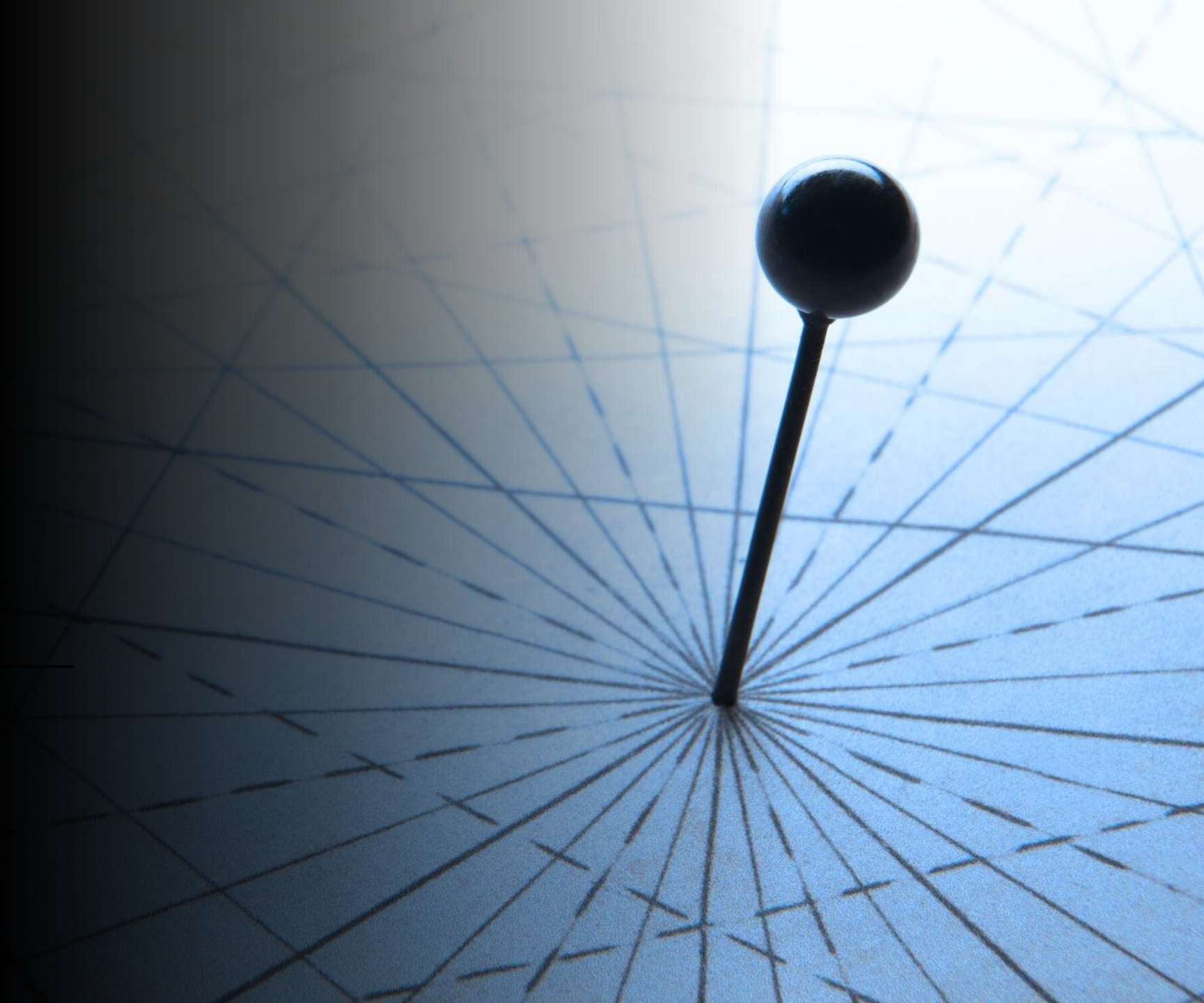




Puntero a función

INFO II
Profesor: Ing. Federico Weber

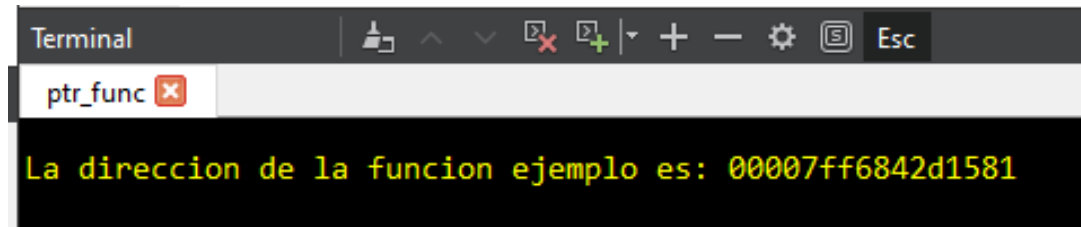


Puntero a función

El identificador [nombre] de las **funciones** es en realidad la **dirección** de memoria donde están alojadas.

```
1  #include <stdio.h>
2
3  int ejemplo()
4  {
5      return 5 ;
6  }
7  int main()
8  {
9      printf("La direccion de la funcion ejemplo es: %p" , ejemplo);
10     return 0;
11 }
12
```

Si lo que quiero es
invocar a la función
debo usar los ()

A terminal window titled 'Terminal' with a tab labeled 'ptr_func'. The output of the program is displayed in yellow text on a black background: 'La direccion de la funcion ejemplo es: 00007ff6842d1581'. The terminal window includes standard navigation and control icons in the title bar.

```
Terminal
ptr_func x
La direccion de la funcion ejemplo es: 00007ff6842d1581
```

La complejidad de los punteros a función es más sintáctica que conceptual

Puntero a función - declaración

¿Cómo declaro un puntero que sirva para guardar la dirección de memoria donde esta alojada la función **ejemplo()** ?

Tipo devuelto Operador * + *Nombre de fantasía que elegimos*

int (*ejemploPtr) () ;

Argumentos

Sin los paréntesis, el operador * se vincula al **int**, no al “identificador”, creando una **función** que devuelve un **puntero a int**, no un **puntero a función**

int *ejemploPtr () ;



Puntero a función - declaración

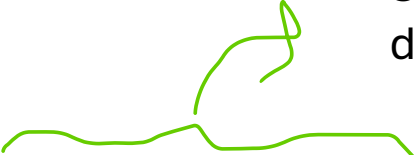
¿Cómo declaro un puntero que sirva para guardar la dirección de memoria donde esta alojada la función **ejemplo()** ?

Si queremos un **puntero const**, debemos colocar “**const**” después del asterisco.
El puntero no se puede cambiar luego de la inicialización.



```
int (*const ejemploPtr) () ;
```

Si colocamos “**const**” antes de los paréntesis, lo que constante es el valor devuelto por la función.



```
int const(*ejemploPtr) () ;
```

Puntero a función - declaración

```
#include <stdio.h>

int ejemplo ()
{
    return 5;
}

int ejemplo2 ()
{
    return 6;
}

int main() {
    int (*ejemploPtr)() = ejemplo;
    printf ("%d\n", ejemploPtr());
    ejemploPtr = &ejemplo2;
    printf ("%d\n", ejemploPtr());
    return 0;
}
```

- Al asignar el **puntero a función** podemos utilizar el **operador &**.
- También funciona si no lo usamos
- Como el **puntero a función** es NO CONSTANTE pudimos re-asignarlo.

```
5
6
Press <RETURN> to close this window...
_
```

Puntero a función - declaración

```
#include <stdio.h>

int ejemplo ()
{
    return 5;
}

double ejemplo2 ()
{
    return 6;
}

int main() {

    int (*ejemploPtr)() = ejemplo;
    printf ("%d\n", ejemploPtr());
    ejemploPtr = ejemplo2;
    printf ("%d\n", ejemploPtr());
    return 0;
}
```

Error/Warning tipo de retorno DISTINTO

**warning: assignment to 'int (*)()' from incompatible
pointer type 'double (*)()' [-Wincompatible-pointer-
types]**

Puntero a función – invocación

```
1  #include <stdio.h>
2
3  int ejemplo(int x)
4  {
5      return x;
6  }
7
8
9  int main()
10 {
11     int (*ejemploPtr)(int) = &ejemplo;
12     int resultado = (*ejemploPtr)(5);
13     printf ("\n%d", resultado) ;
14
15     resultado = ejemploPtr(6);
16     printf ("\n%d", resultado);
17
18     getchar();
19     return 0;
```

Declaramos el **puntero a función** (tener en cuenta el **argumento**)

Modo explícito de llamar a la función a través del puntero

Modo implícito de llamar a la función a través del puntero

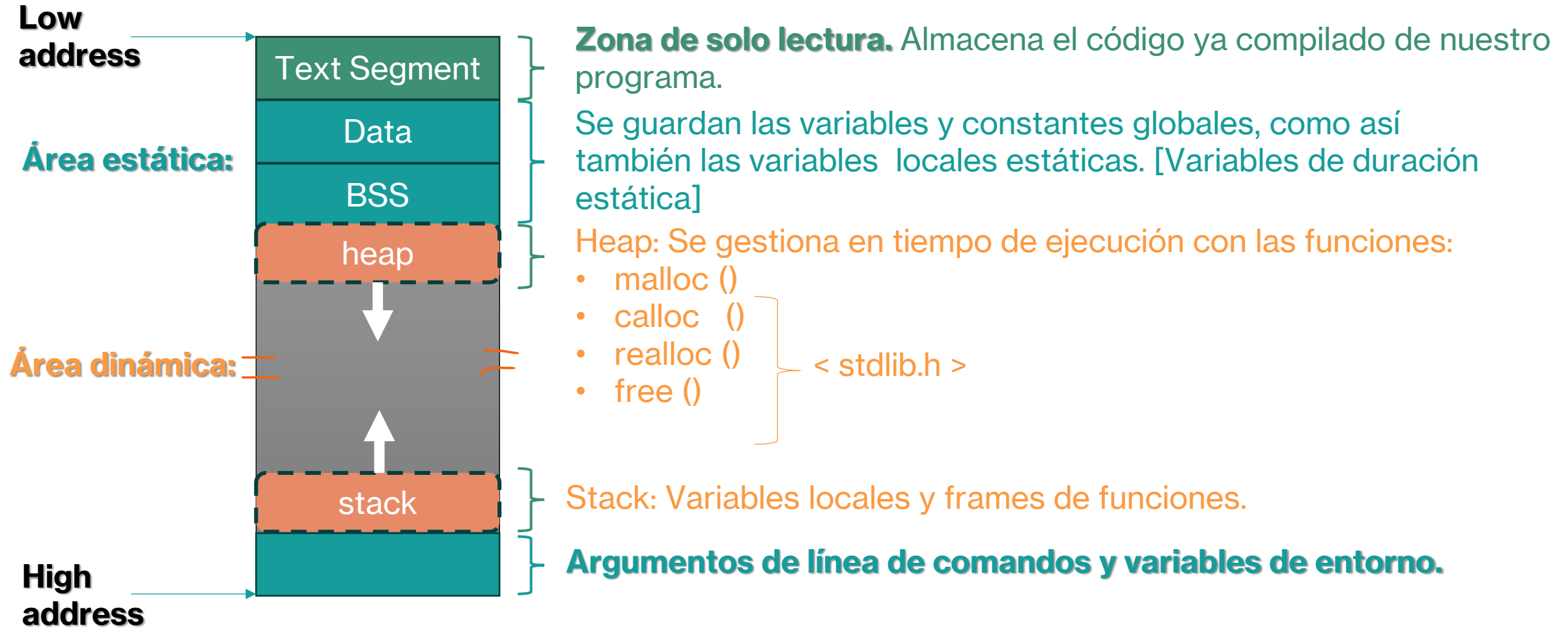
Puntero a función – Ejemplos IDE

- 1) ptr_fun1 -> Mostrar argumentos en la declaración y formas de llamar a la función a través del puntero
- 2) ptr_fun2 -> ídem 1
- 3) ptr_fun3 -> ver warning en printf
- 4) Ptr_fun4 -> solucionar warning
- 5) Ptr_fun5 -> uso de typedef

```
int printf(const char *format, ...);
```


Segmentación de memoria - repaso

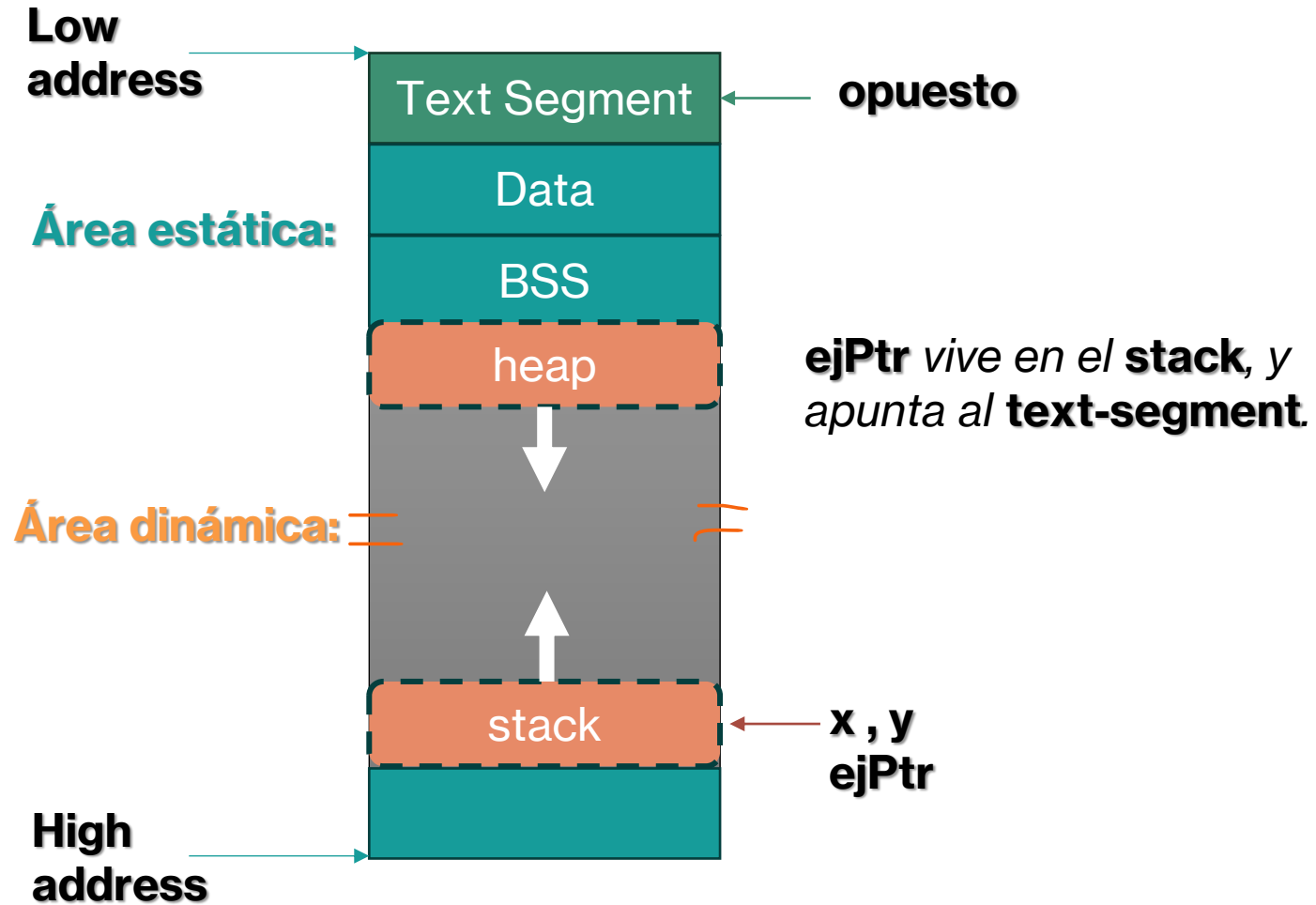
Un programa en C, al ejecutarse, divide su espacio de memoria en **segmentos** bien definidos.



No se puede modificar el text segment durante la ejecución.

Segmentación de memoria

Un programa en C, al ejecutarse, divide su espacio de memoria en **segmentos** bien definidos.



```
#include <stdio.h>

int opuesto (int a)
{
    return -a;
}

int main()
{
    int (*ejPtr)(int) = opuesto;
    int x=5, y;

    y = ejPtr (x);
    printf ("y es: %d\n", y);

    return 0;
}
```

No se puede realizar aritmética de punteros con punteros a función

The background of the slide features a blurred image of three red location pins standing on a dark, winding road. The pins are arranged in a line that recedes into the distance, with the middle pin being the most prominent and in focus. The background is a soft, out-of-focus blue and white, suggesting a bright sky.

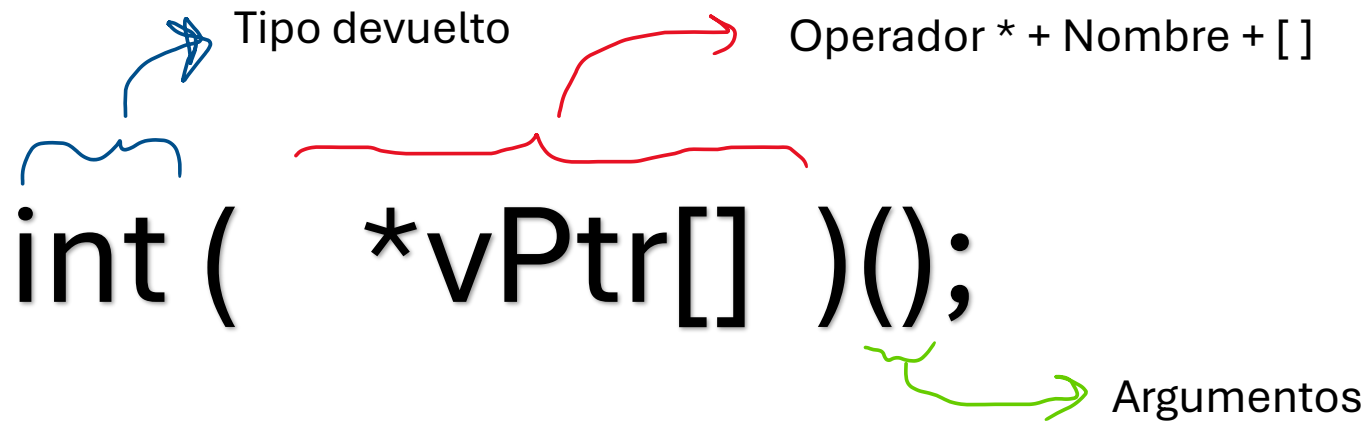
Aplicaciones de los punteros a funcion

INFO II

Profesor: Ing. Federico Weber

Vector de punteros a función [tablas de función]

- Un caso de uso para los punteros a función es la construcción de **tablas de función** o **dispatch tables**.
- Esto consiste en un vector que almacena múltiples punteros a función, típicamente para poder seleccionar y ejecutar funciones según alguna **clave** o **índice**.
- Es una alternativa eficiente a usar múltiples **if/else** o **switch** para decidir qué función ejecutar.
- Permite soluciones eficientes para **menús**, **maquinas de estado** o cuando queremos reemplazar algoritmos de forma sencilla.
- Es una forma de imitar el **polimorfismo**


`int ( *vPtr[] )();`
Tipo devuelto Operador * + Nombre + [] Argumentos

- 1) vptr1 -> opción al switch case. Mostrar error.
- 2) vptr2 -> mostrar la solución al error.
- 3) Vptr3 -> con un for

Pasar funciones como argumentos [callbacks]

- Un **callback** es simplemente una función que se pasa como parámetro a otra función para que ésta la invoque en algún momento.
 - Esto permite, por ejemplo, que una función genérica pueda llamar a código específico provisto por el usuario.
-
- 1) Callback1 -> lo hacemos a mano para mostrar paso a paso. Programa que elije entre dos funciones, la condición de la elección se hace en tiempo de ejecución
 - 2) suma_2_3 -> Programa que devuelve la suma del cuadrado de dos números, o de los cubos de dos números
mostrar detalladamente prototipo de función y usamos callback junto a una tabla de funciones

Punteros a función en estructuras [simulación de métodos]

El lenguaje C no tiene soporte directo para organizar el código en "objetos" como en otros lenguajes más modernos [C++ por ejemplo].

Sin embargo, podemos **agrupar datos y funciones que trabajan con esos datos** usando una herramienta que sí tiene C: las estructuras y los punteros a función.

Esto significa que **dentro de una estructura** podemos guardar no solo variables, sino también **direcciones de funciones**, es decir, funciones que luego pueden ejecutarse desde adentro de esa estructura.

Esta técnica es muy útil cuando queremos **organizar mejor nuestro programa**, por ejemplo, para que diferentes partes del código trabajen de forma parecida pero con comportamientos distintos. Es algo que se usa en proyectos grandes escritos en C.

En resumen, **una estructura con punteros a función** permite que cada "grupo de datos" tenga sus propias funciones

1) método_en_c.c -> figura geométricas, calculo de área de circulo y rectangulo

Calculo numérico

INFO II

Profesor: Ing. Federico Weber

Calculo numérico

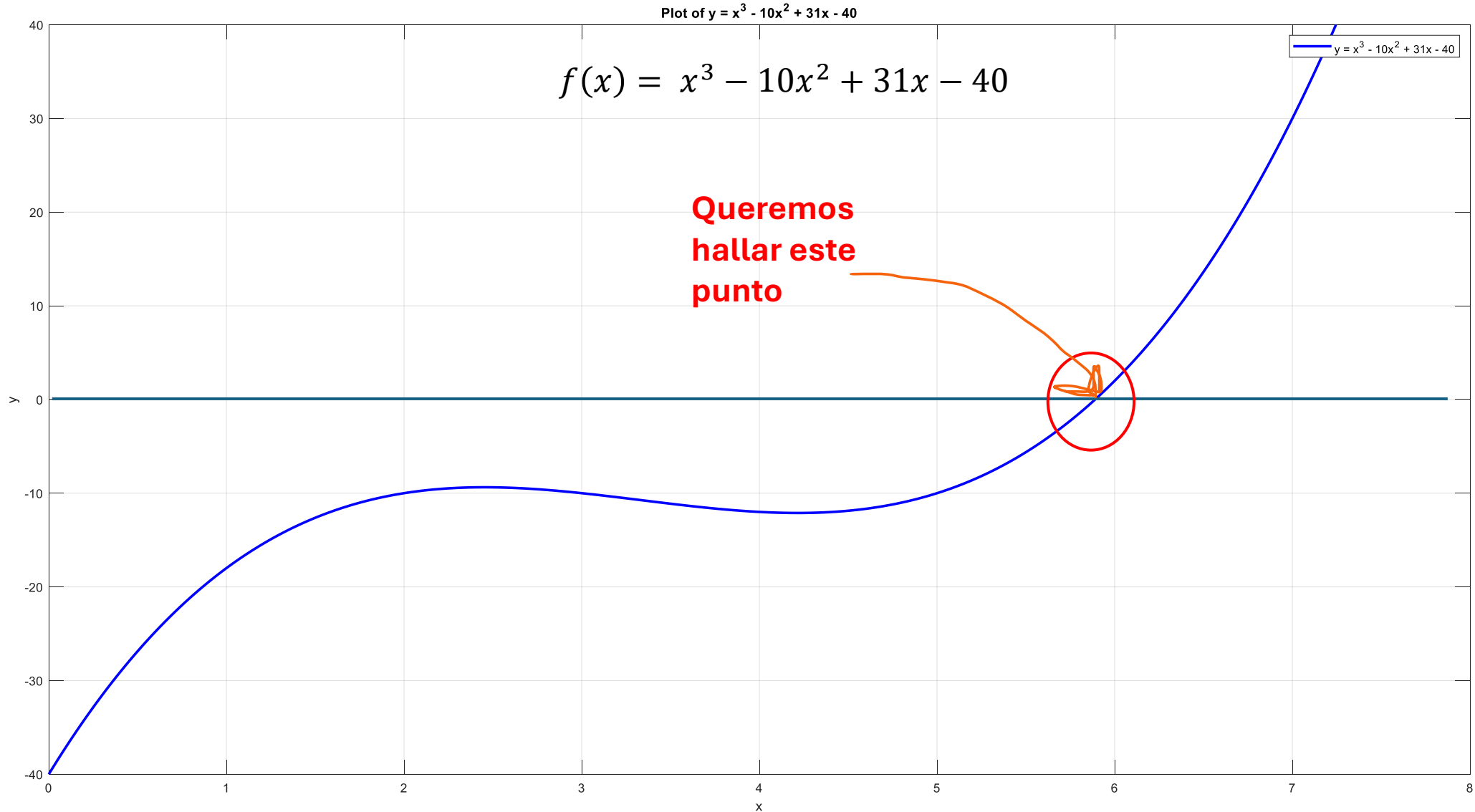
- El cálculo numérico es una rama de las matemáticas y la computación que se enfoca en la *aproximación* de soluciones a problemas matemáticos utilizando *métodos algorítmicos*.
- En lugar de buscar soluciones exactas, como se hace en el cálculo analítico, *el cálculo numérico se centra en encontrar soluciones aproximadas que sean suficientemente precisas para aplicaciones prácticas.*

Determinación del cruce por cero – Regula Falsi (regla falsa)

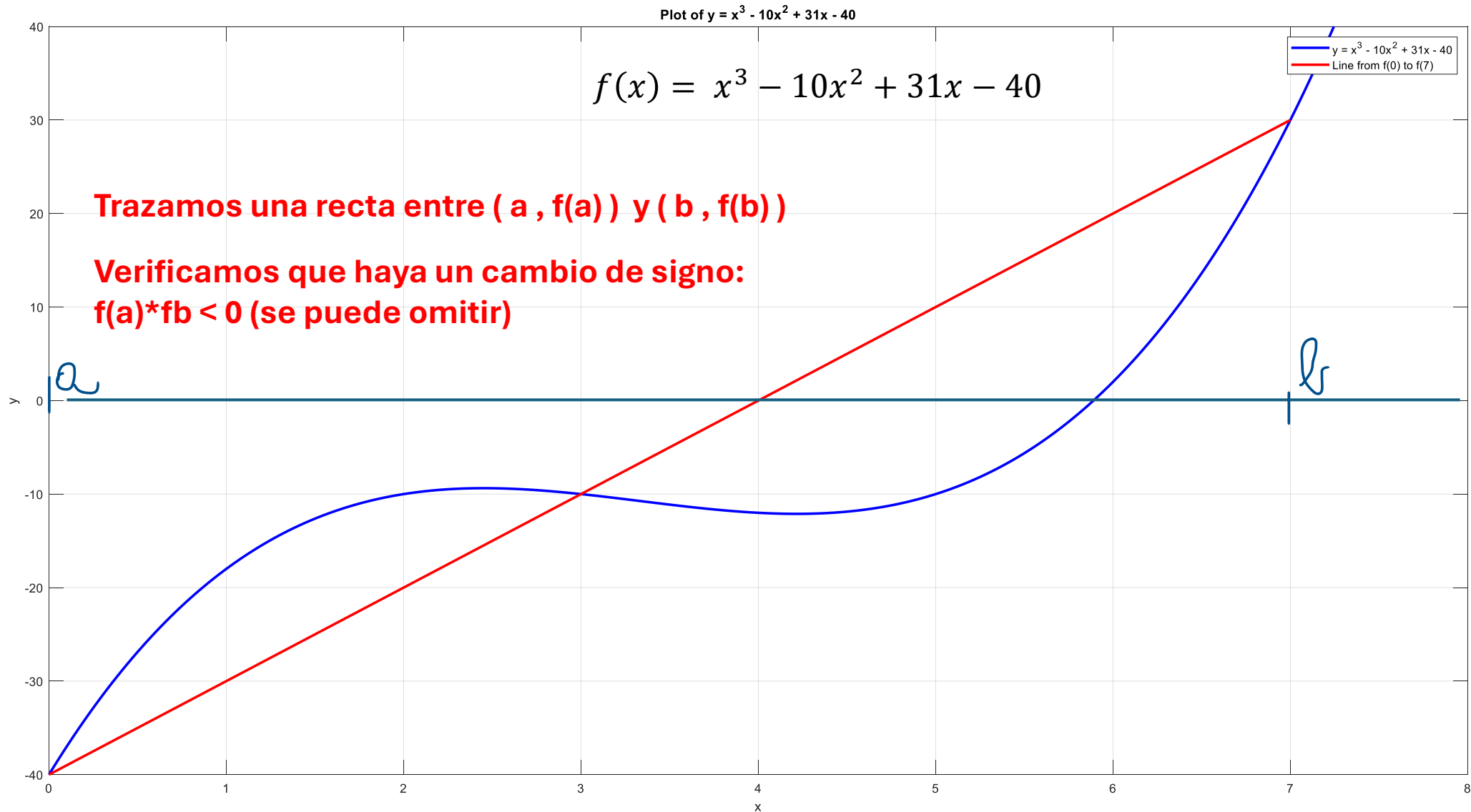
- Se parte de una función $f(x)$ y de un intervalo $[a,b]$ donde la función cambia de signo.
- Vamos a usar como ejemplo la siguiente función:

$$f(x) = x^3 - 10x^2 + 31x - 40$$

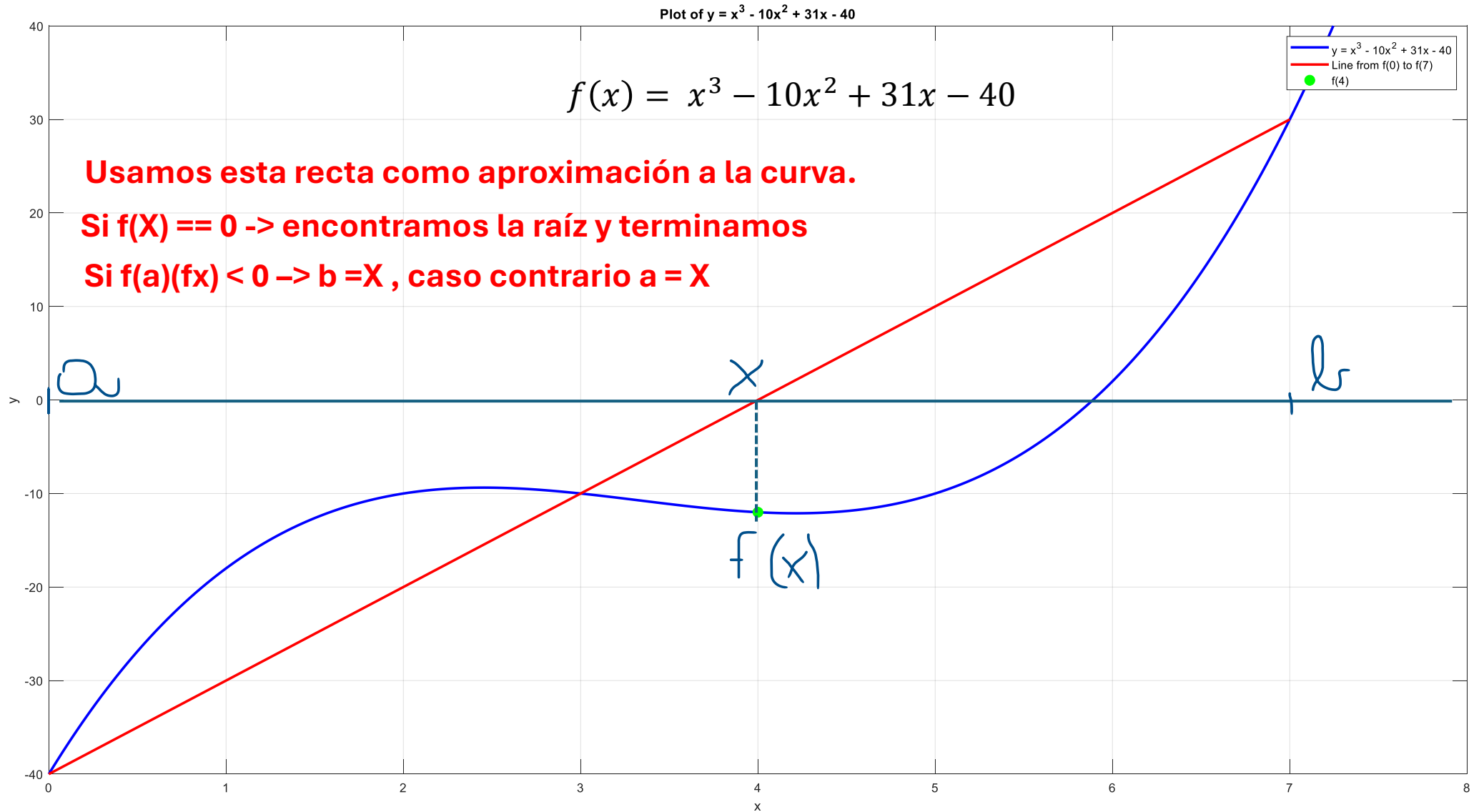
Determinación del cruce por cero – Regula Falsi (regla falsa)



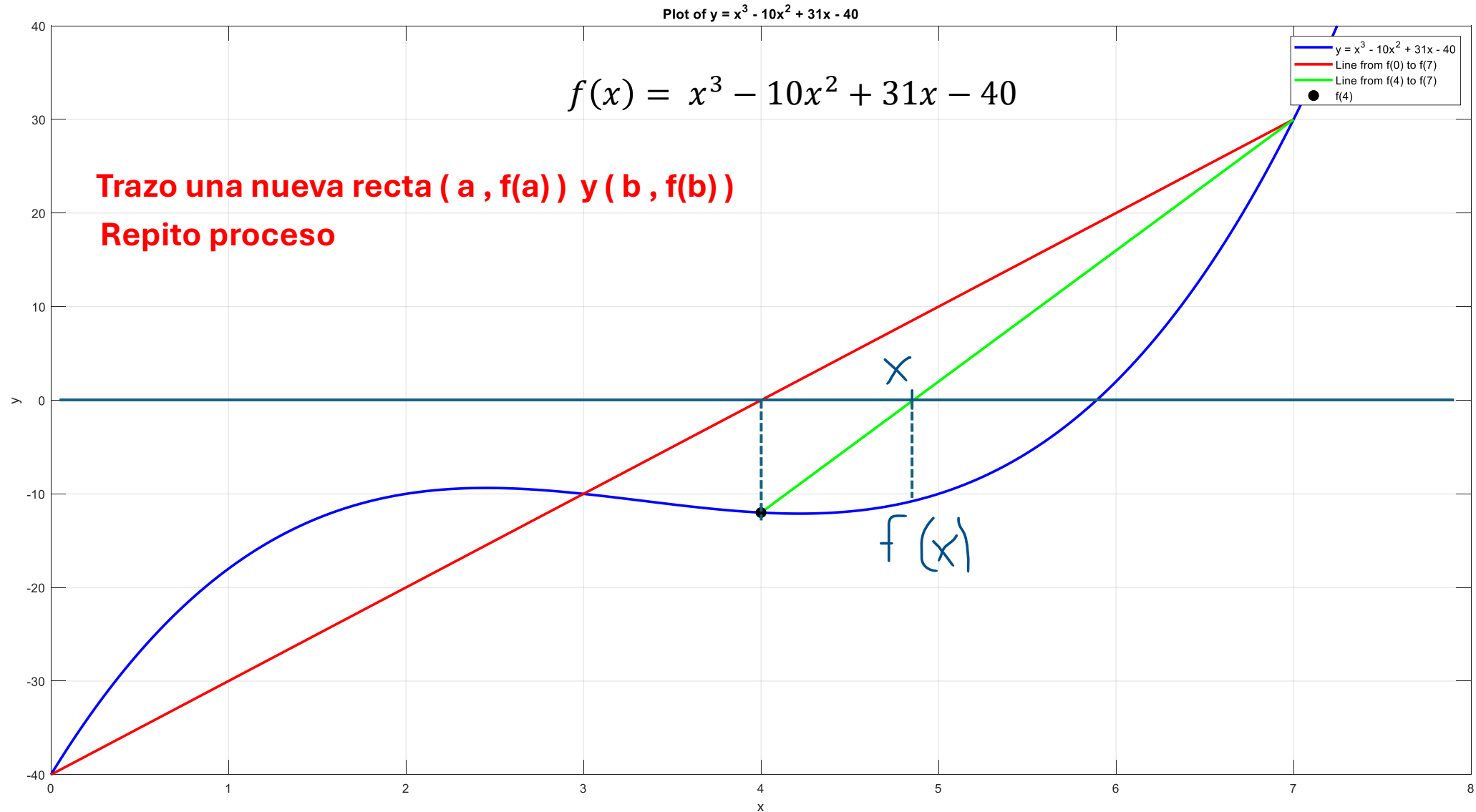
Determinación del cruce por cero – Regula Falsi (regla falsa)



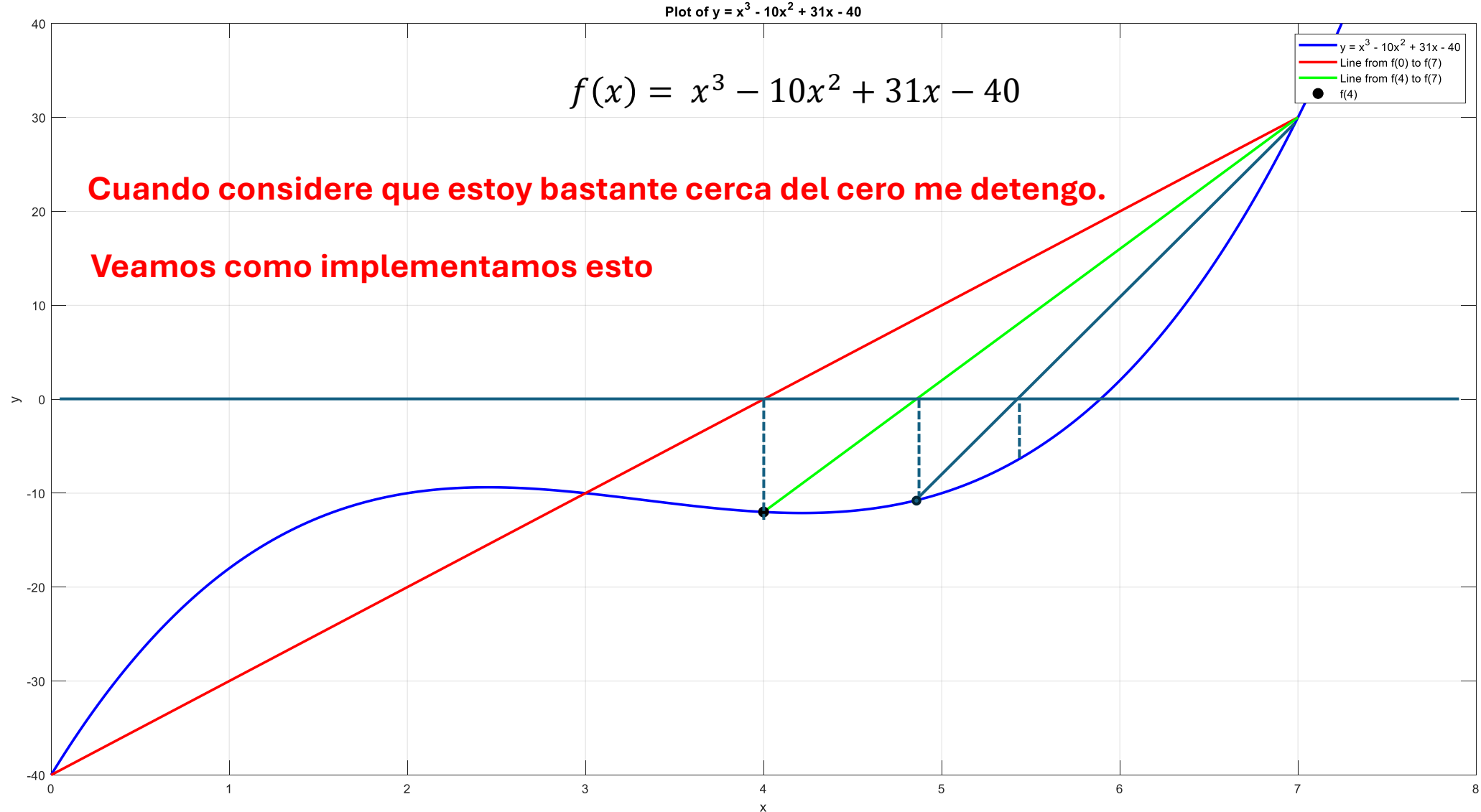
Determinación del cruce por cero – Regula Falsi (regla falsa)



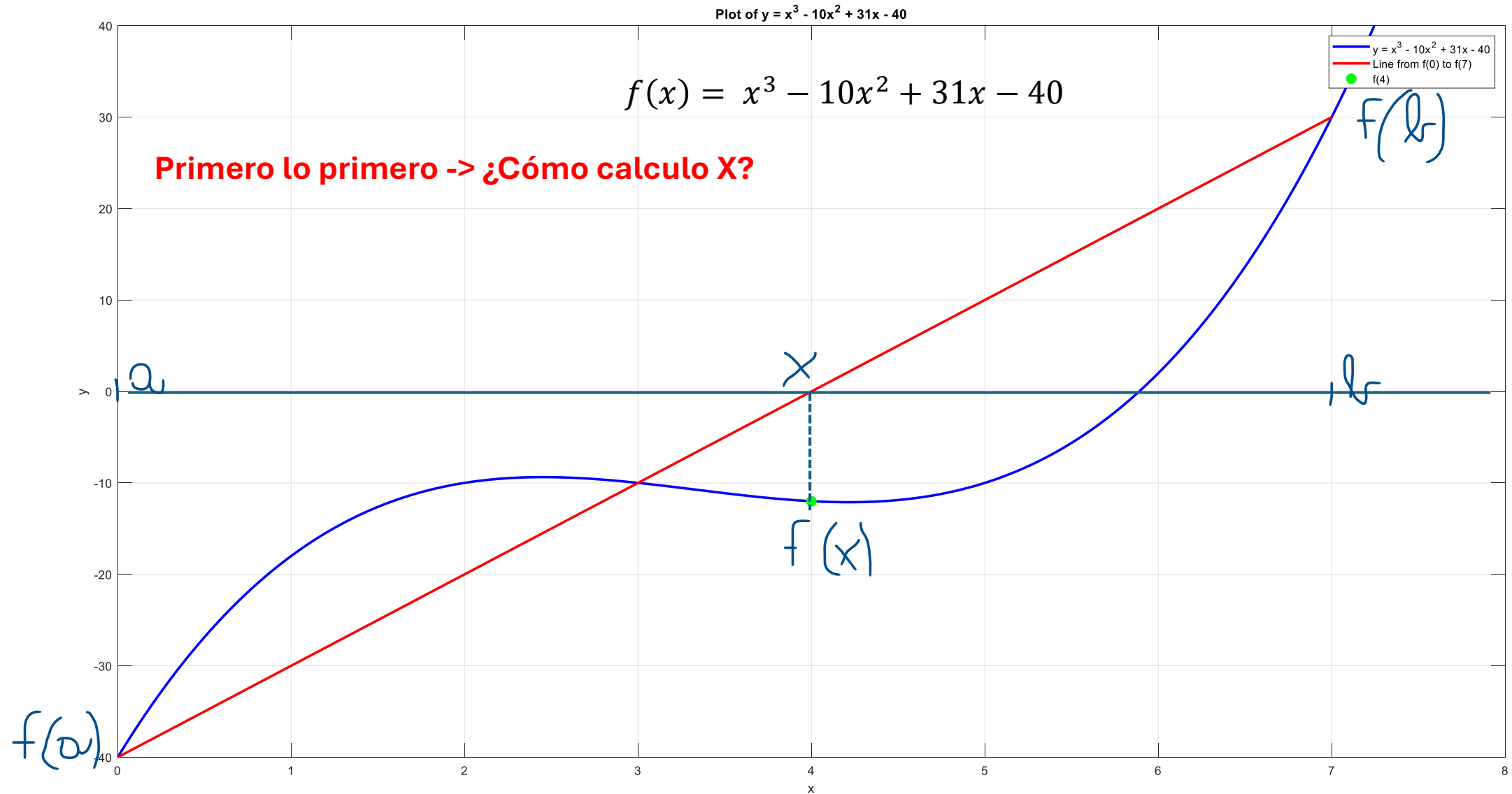
Determinación del cruce por cero – Regula Falsi (regla falsa)



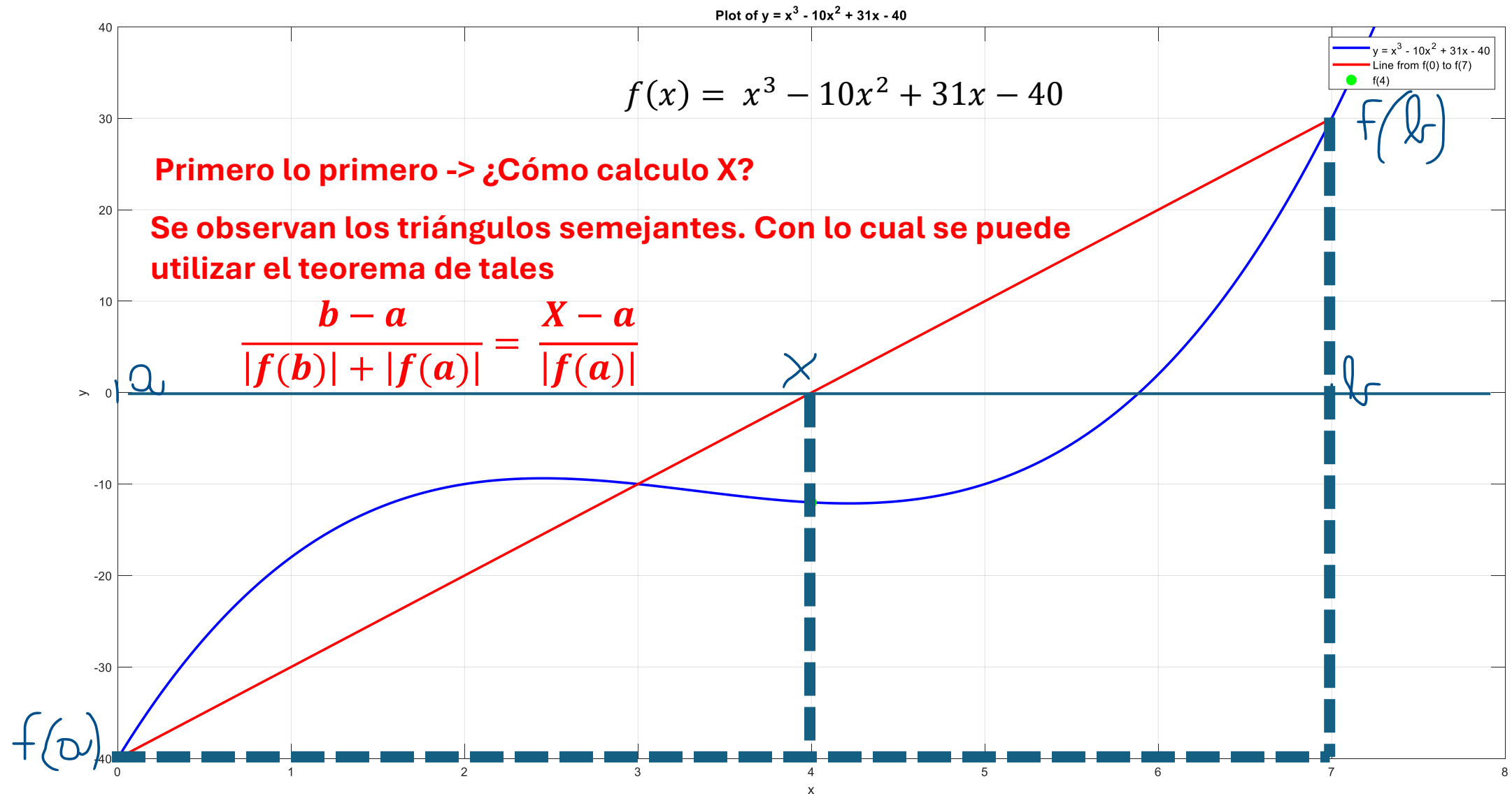
Determinación del cruce por cero – Regula Falsi (regla falsa)



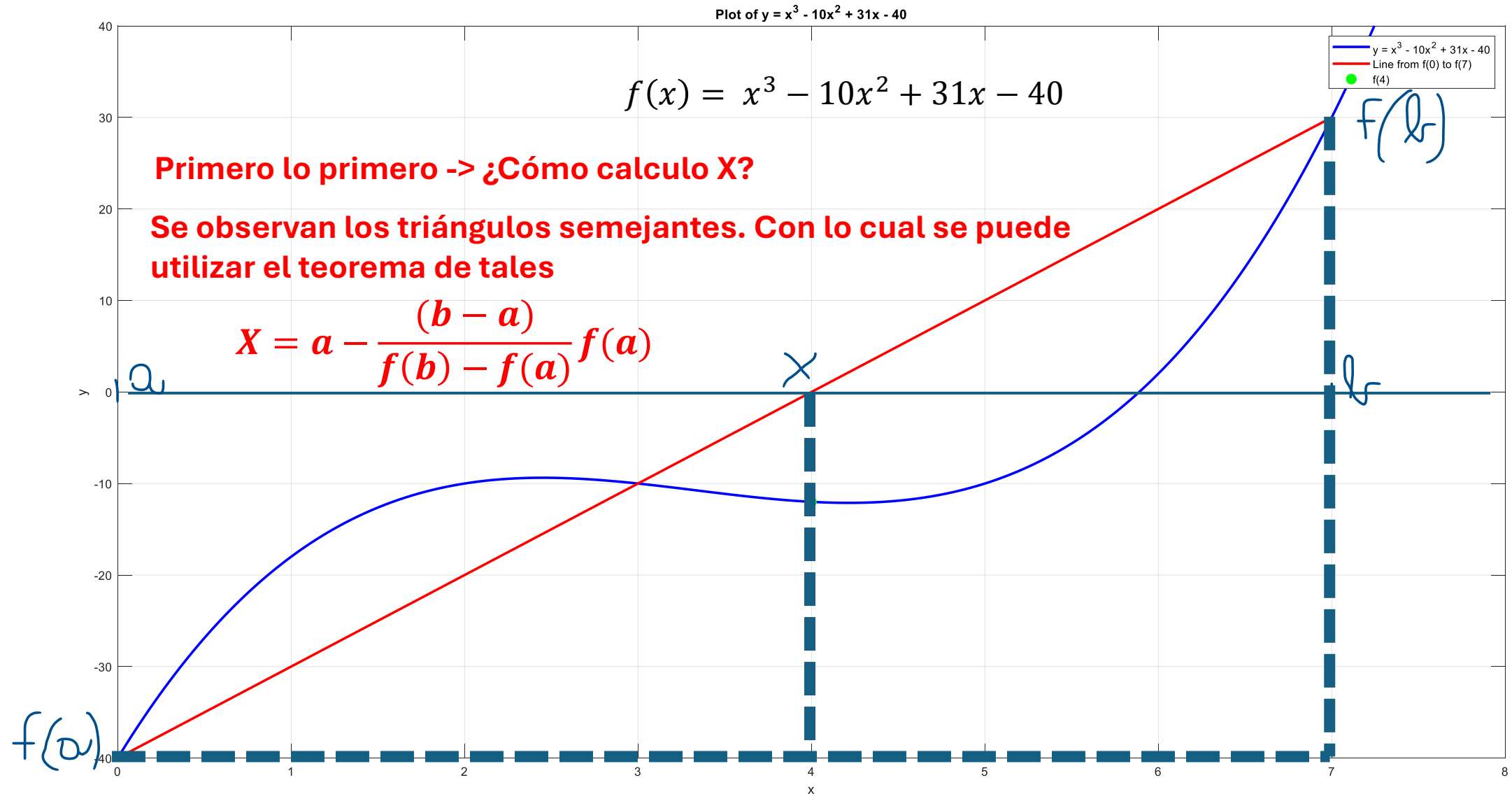
Determinación del cruce por cero – Regula Falsi (regla falsa)



Determinación del cruce por cero – Regula Falsi (regla falsa)



Determinación del cruce por cero – Regula Falsi (regla falsa)



Determinación del cruce por cero – Regula Falsi (regla falsa)

Vamos a implementar el algoritmo de dos formas diferentes:

1. Forma iterativa

2. Forma recursiva

Regula_falsi.c

**Veamos una forma
de buscar los
intervalos sin salir
de C en una
función
desconocida**





FIN